Applied Software Development Project

Software Development and Integration for a Novelty Similarity Detection Model

Assignment 3

Alisher Tokesh | CSE-2507M

Continuation of Assignments 1 and 2

9 October 2026

Contents

1 Project context and development process 3

2 Selection of technologies and tools 4

3 Version control and GitHub collaboration 5

4 Continuous integration and delivery 6

5 Scientific software development 7

6 Practical module and repository evidence 8

7 Conclusions and recommendations 9

References 9

Executive summary

This report continues the same thesis-topic novelty screening project assessed in Assignments 1 and 2. Assignment 1 established a 17-week critical path, USD 38,188 in direct cost, a USD 5,728 contingency reserve, and a USD 43,916 planned budget. Assignment 2 recommended stage-gated iterative research and development and a modular monolith with explicit adapters. The present assignment turns those choices into a version-controlled scientific software module with a public repository, automated tests, an issue workflow, and a working CI pipeline.

The module accepts a proposed title, abstract, keywords, domain, and language. It combines semantic similarity, keyword uniqueness, and domain rarity into an explainable novelty assessment. The current prototype uses in-memory retrieval; the planned pgvector integration is tracked as a future task. A high similarity result is a reviewer prompt, not an automatic academic rejection.

The public repository is github.com/Tsarkashrk/novelty-similarity-detection-model. The tested workflow is GitHub Actions run 37891050872.

1 Project context and development process

A software process converts a research idea into a maintainable service through defined requirements, design, implementation, verification, release, and feedback. In this project, each stage produces evidence that an academic reviewer can inspect. The process is iterative because multilingual retrieval and threshold choices depend on corpus quality and labelled examples; it is stage-gated because student-facing decisions require explicit approval of data access, model quality, security, and pilot readiness.

Stage	Output	Decision evidence
Requirements and data	Topic fields, duplicate policy, data access approval	Approved scope and labelled-pair protocol
Design and implementation	Hybrid scorer, API contract, repository boundary	Runnable vertical slice and code review
Verification	Unit tests, integration plan, quality and latency measurements	F1, near-duplicate recall and error samples
Pilot and release	Reviewer workflow, training and monitored deployment	Human-review record and rollout decision

Table 1 Deliverables and evidence across the 17-week plan

The critical path from Assignment 1 runs through requirements (A), data approval (B), baseline adaptation (D), backend integration (F), coordinator workflow (G), QA (H), pilot (I), deployment (J), and closure (K). Corpus preparation (C) and calibration (E) have limited slack, so their work can be refined in short cycles without losing sight of the fixed finish date. Weekly reviews should compare the Gantt baseline with actual progress and log decisions that consume reserve funds.

Responsibilities follow the WBS: the academic lead owns the duplicate policy and final approval; the data owner authorizes and validates the corpus; ML and backend engineers implement the model and service; QA and domain reviewers test behavior and examine false high-risk flags. A definition of done for each increment includes a passing test, versioned inputs, an explanation of results, and a named reviewer for any academic-risk decision.

2 Selection of technologies and tools

Technology selection should be driven by functional fit, scientific reproducibility, team skill, integration cost, operating cost, security, and the ability to replace a component later. The 17-week schedule and USD 43,916 cap favor a small deployable service. A technology earns its place when it makes a measurable requirement easier to satisfy; novelty alone does not justify additional infrastructure.

Choice	Rationale	Trade-off and control
Python 3.10+	Existing code, NLP ecosystem, typed dataclasses and simple packaging	Runtime speed is lower than compiled languages; benchmark hot paths
FastAPI	Existing assessment endpoint and machine-readable API boundary	Add platform authentication and contract tests before pilot
Hashing backend	Deterministic and dependency-light CI baseline	Not a claim of multilingual semantic quality
Multilingual E5	Multilingual embeddings for Russian, Kazakh and English topics	Model download, compute cost and language-specific evaluation
PostgreSQL + pgvector	Planned persistent top-k search using an existing schema	Current prototype remains in memory; adapter and load tests are pending

Table 2 Technology decisions against project goals

The packaging file *pyproject.toml* keeps the core dependency-free and groups API, embedding, experiment, and development tools as optional extras. This supports two environments: a small deterministic CI environment for fast feedback and a controlled research environment with a pinned embedding model and labelled validation data. The boundary around *EmbeddingBackend* makes it possible to compare encoders without rewriting the scoring policy.

Before changing the model or database, the team should record corpus version, language distribution, embedding model identifier, vector dimension, thresholds, and evaluation results. That record connects engineering changes to the KPIs from Assignment 2: duplicate F1 at least 0.82, near-duplicate recall at least 0.75, false high-risk rate at most 8 percent, and top-k latency at most 1.5 seconds. These are pilot targets, not measured achievements of the current CI run.

3 Version control and GitHub collaboration

Git records source changes as commits in a distributed history. It supports local branches, reviewable differences, and recovery of earlier versions. GitHub hosts the Git repository and adds pull requests, issues, access controls, and automation; it is a collaboration platform rather than a separate version control algorithm. Subversion (SVN) is a useful comparison because it keeps the authoritative history on one central server. For a research module with experiments and a small team, Git's cheap local branches and reproducible commit identifiers are a strong fit.

Capability	Git	SVN	GitHub platform
History	Complete local clone	Central server	Remote Git hosting
Offline work	Commit and inspect locally	Limited without server	Depends on Git locally
Review	Diff and merge tools	Diff and merge tools	Pull requests and checks
Project tasks	No built-in issue tracker	No built-in issue tracker	Issues and templates

Table 3 Version control and collaboration comparison

The repository uses a *main* branch and a public remote. A normal change should begin with a focused issue, use a short-lived branch, include a test or reproducible example, and be reviewed before merge. The README documents local setup and the meaning of the CI badge. The MIT license states reuse terms. Two issue templates request reproducible bug evidence or explicit acceptance evidence for research tasks, while warning contributors not to post private student data.

For this assignment, commit **897960e** published the workflow, license, issue templates, README update, and CLI test. Issue 1 tracks the future pgvector repository adapter with integration-test criteria. This keeps unfinished production work visible without presenting it as implemented. Git tags can later mark a validated release, with the corresponding model configuration and corpus version recorded alongside it.

4 Continuous integration and delivery

Continuous integration (CI) runs automated checks whenever changes enter the shared history. It reduces the delay between introducing an error and finding it. Continuous delivery (CD) prepares a tested release candidate so a human can approve deployment; continuous deployment additionally releases each passing change automatically. Academic approval and private-data access make a manual promotion gate appropriate for this pilot.

Trigger	Automated gate	Output
Push to main or pull request	Install package, run unit and CLI tests on Python 3.10 and 3.12; compile source	Pass or fail status visible on GitHub
Successful push to main	Build a wheel only after both test jobs succeed	Versioned package candidate
Successful package job	Upload the wheel as a workflow artifact	Downloadable artifact for manual review

Table 4 Implemented CI and delivery-candidate path

The YAML workflow is stored at .github/workflows/ci.yml. Its job dependency enforces the order: test first, package second. The token is restricted to read-only repository contents. This avoids a workflow that can publish or deploy without approval. A dependency cache speeds repeated Python jobs, while the wheel artifact preserves the exact build output of a successful run.

The first public run, run 37891050872, completed successfully for commit 897960e. The Python 3.10 test job, Python 3.12 test job, and package job all passed, and the *thesis-novelty-wheel* artifact was uploaded. Locally, the same three automated tests passed and a wheel was built. These checks prove packaging and baseline behavior; they do not prove multilingual model quality, secure institutional access, or production latency. Those remain explicit quality gates in the 17-week project plan.

5 Scientific software development

Scientific software must make a claim reproducible. For a novelty detector, a score is meaningful only with its input corpus, preprocessing rules, embedding model, weights, thresholds, and labelled evaluation protocol. The requirements therefore combine ordinary software behavior (valid API responses and stable packaging) with research behavior (traceable experiments, error analysis, and performance by language). A passing test suite cannot substitute for a representative validation set.

Environments and libraries

The core Python package uses standard-library dataclasses and a deterministic hashing embedding backend. This keeps CI fast and allows tests to run without a large model download. FastAPI and Pydantic are optional service dependencies; SentenceTransformers supplies research-grade multilingual embeddings when enabled; pandas, scikit-learn, and matplotlib are optional experiment tools. A production-like environment must pin exact versions and model identifiers, record the corpus revision, and prohibit real student records in public fixtures.

Verification beyond unit tests

Unit tests check duplicate and unrelated-topic behavior. The CLI smoke test checks the public command, JSON structure, and explanation components with synthetic sample data. The next layer should test an API contract, a pgvector repository adapter, migration and restart behavior, and retrieval agreement against the in-memory baseline. Evaluation should report precision, recall, F1, false high-risk rate, and calibration by Russian, Kazakh, and English topic subsets. Timing tests should use the expected pilot corpus rather than the ten-row sample CSV.

The scientific risk is a false sense of validity: a deterministic baseline may pass CI while semantic quality remains weak. The mitigation is to freeze a labelled benchmark before tuning, compare baselines with multilingual embeddings, keep an error log for disputed cases, and require a human reviewer before any high-risk topic is rejected. Such practices make the research result testable and align the code with the academic governance defined in Assignments 1 and 2.

Reproducibility controls

Each experiment should record the Git commit, dependency versions, model identifier, corpus hash, language mix, threshold settings, and exact evaluation script. Data should be split before tuning so near-duplicate pairs from the same topic family do not leak into both development and test sets. A failed quality gate should produce an issue with a synthetic reproduction case and a named owner, then be rechecked against the unchanged benchmark.

6 Practical module and repository evidence

The practical module is the existing *thesis_novelty* package, now documented and checked by CI. A CSV loader creates historical topic records. Preprocessing and an embedding backend encode query and corpus text. The model ranks candidates by cosine similarity, calculates keyword uniqueness and domain rarity, combines the three components with default weights 0.60, 0.25, and 0.15, and returns a label plus explanation. The CLI and FastAPI adapter expose the result. Default similarity cutoffs are 0.90 for duplicate and 0.75 for near-duplicate.

Repository component	Current function	Assignment 2 boundary
model.py	Hybrid scoring and in-memory top-k ranking	Keep scoring independent from storage
embeddings.py	Interchangeable hashing and SentenceTransformer backends	Replace model without changing use case
api.py and cli.py	Service and command-line adapters	Expose a stable assessment contract
test_model.py and test_cli.py	Behavior and command-line checks	Extend to storage and API integration

Table 5 Small scientific module and intended evolution

The public repository includes README instructions, the MIT license, two issue templates, the CI workflow, sample topics, labelled pairs, source code, tests, and the pgvector schema. The Actions run provides independent evidence that the committed state passes its automated gates. The pgvector issue records a concrete next step with acceptance tests. These links let an instructor inspect both the software and the process.

The practical work deliberately stops at a delivery candidate. It does not claim that pgvector is connected, that student data has been approved, or that the university platform is deployed. These gaps correspond to WBS work packages 1.2, 1.4, 1.5, and 1.6 from Assignment 1 and to the ports and quality attributes assessed in Assignment 2.

7 Conclusions and recommendations

The assignment produced a public, version-controlled scientific module with documented technology choices, a license, issue tracking, automated tests, and a successful CI run that builds a wheel. Python, FastAPI, replaceable embedding backends, and a future pgvector adapter fit the project because they preserve explainability and allow the team to test each uncertain component. The GitHub workflow gives immediate integration feedback and a candidate package while retaining the manual release decision required by academic review.

One challenge encountered was that a title-only CLI test did not reproduce an exact duplicate: the model also embeds the abstract, keywords, and domain. The test was corrected to provide the complete public fixture, which makes the check reflect the actual input contract. Another boundary is that the current three-test suite and hashing fallback are engineering smoke checks, not the multilingual validation required for the pilot. The next increment should use the labelled corpus and per-language metrics, then add storage and API integration tests before deploying to coordinators.

Recommendation: keep the 17-week and USD 43,916 baselines from Assignment 1, use the stage-gated iterative method and modular boundaries from Assignment 2, and require passing quality, security, latency, and human-review gates before production rollout. The current CI result supports continued development; a release decision still depends on measured model and workflow outcomes.

References

1. Tokesh, A. Assignment 1: Project Management and Financial Justification for a Novelty Similarity Detection Model. Course report, 2026.

2. Tokesh, A. Assignment 2: Development Methodology and Architecture Assessment for a Novelty Similarity Detection Model. Course report, 2026.

3. Hybrid Thesis Topic Novelty Assessment Model. Public repository, commit 897960e; accessed 9 October 2026.

4. Chacon, S. and Straub, B. Pro Git, chapter 1.1, About Version Control.

5. GitHub Docs. Understanding GitHub Actions; Workflow artifacts.

6. Python Packaging Authority. The Packaging Flow.